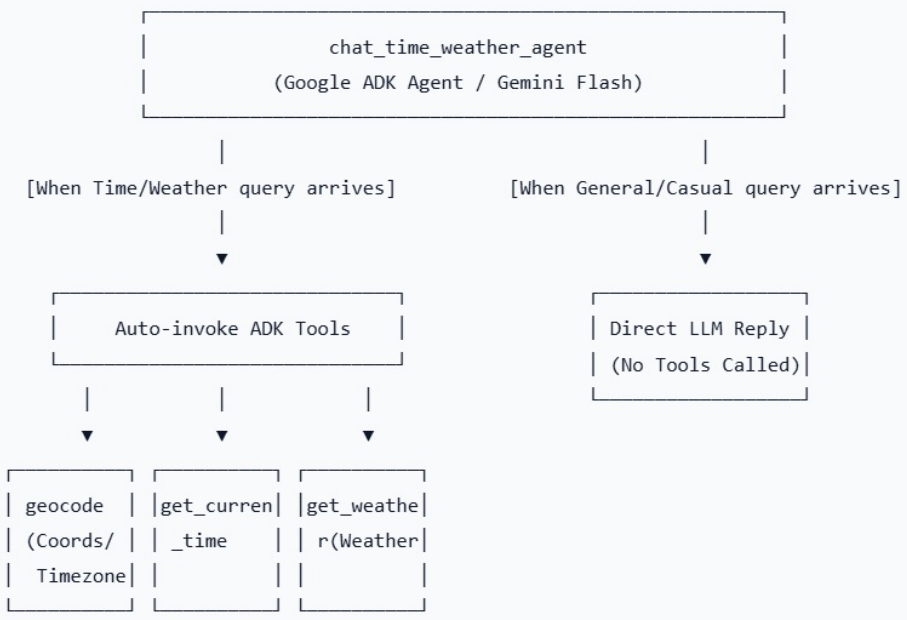
chat_time_weather_agent — Time, Weather, and General Chat Agent

`chat_time_weather_agent` is a versatile assistant agent built on top of the Google Agent Development Kit (ADK). It is equipped with dedicated tools to query real-time local time and meteorological information for major cities worldwide, while also seamlessly handling general knowledge questions, programming queries, and casual conversations without invoking external tools.

> 💡 **Folder Naming Convention and ADK Specification**: In compliance with standard Python package naming conventions (PEP 8) and the Google ADK `AgentLoader` regular expression specification (`^[a-zA-Z0-9_]+$`), this agent is structured in a single folder named `chat_time_weather_agent` using underscores (`_`) instead of hyphens (`-`). This fundamentally prevents Python module `import` syntax errors and `adk run` command loading failures.

---

1. Key Features and Architecture



1. **Intelligent Geocoding (`geocode`)**:
    - Calls the Open-Meteo Geocoding API to resolve incoming city or region names into latitude, longitude, and standard IANA timezone identifiers (e.g., `Asia/Seoul` , `America/New_York` , `Europe/Paris`).
    - Includes a built-in dictionary mapping major Korean city names (Seoul, Busan, Jeju, Tokyo, New York, Paris, London, etc.) to English names, ensuring high search accuracy.
    - Embeds an offline fallback database to guarantee operational continuity during network failures or API timeouts.
2. **Precise Local Time Calculation (`get_current_time`)**:
    - Applies Python's `zoneinfo.ZoneInfo` based on the standard IANA timezone resolved via geocoding, accurately calculating the actual local time (including UTC offset and timezone abbreviation).
3. **Real-time Weather Query (`get_weather`)**:
    - Calls the Open-Meteo real-time weather API to fetch WMO weather codes (automatically translated into human-readable descriptions), temperature, feels-like temperature, relative humidity, and wind speed in real time.
4. **General Conversation and Knowledge Q&A (General Chat Routing)**:
    - Guided by carefully designed instruction prompt engineering, general queries that do not require time or weather information (programming, summarization, translation, common knowledge, etc.) bypass tool calls entirely, allowing the model to respond directly and rapidly.

---

2. File Structure

```
chat_time_weather_agent/
├── __init__.py          # Exposes agent module and root_agent object
├── agent.py             # Defines geocode, get_current_time, get_weather tools and root_agent
├── README-KR.md         # Korean detailed documentation
└── README-EN.md         # English translated documentation (this document)
```

- `agent.py` :
  - `geocode(location: str) -> dict` : Resolves city names into coordinates and timezones
  - `get_current_time(location: str, latitude: float, longitude: float) -> dict` : Returns real-time local time
  - `get_weather(location: str, latitude: float, longitude: float) -> dict` : Returns current temperature, humidity, feels-like temperature, and weather condition
  - `root_agent` : Google ADK `Agent` instance (including model, prompt instructions, and tool registry)
- `__init__.py` :
  - Exposes `from .agent import root_agent` as the entrypoint recognized by the ADK CLI

---

3. Prerequisites and Configuration

1) Python Environment and Package Installation

Install ADK and required libraries in a Python 3.10+ environment:

```
pip install google-adk python-dotenv
# or using uv
uv pip install google-adk python-dotenv
```

2) Environment Variables (`.env`) Configuration

Create a `.env` file in the parent folder (`part02_architecture` or project root) and configure your Gemini API key:

```
# When using Gemini Developer API Key
GEMINI_API_KEY=your_gemini_api_key_here

# Or when using Google Cloud Vertex AI
# GOOGLE_GENAI_USE_VERTEXAI=TRUE
# GOOGLE_CLOUD_PROJECT=your-gcp-project-id
# GOOGLE_CLOUD_LOCATION=us-central1

# Optional: Specify model to use (default: gemini-flash-latest)
ADK_MODEL=gemini-flash-latest

# Optional: Force enable offline testing mode (0: online, 1: offline built-in data)
AGENT_OFFLINE=0
```

---

4. Execution and Testing Methods

Method 1: Terminal Interactive CLI Mode (`adk run`)

The most intuitive way to directly converse with the agent in your terminal.

Run inside the `adk-based_Agent` directory:

```
# From within the adk-based_Agent folder
adk run chat_time_weather_agent
```

Or run from the parent `part02_architecture` directory:

```
cd ..
adk run adk-based_Agent/chat_time_weather_agent
```

Once the prompt appears, type your questions:

```
User: What time is it in Paris right now?
User: Tell me the weather in Seoul.
User: What is a generator in Python?
```

Method 2: Web Development UI Environment (`adk web`)

Visually inspect conversations, tool execution parameters, event streaming, and session states through the official browser GUI provided by Google ADK.

```
# Run from within the adk-based_Agent directory
adk web --port 8008
```

After starting, open your browser and navigate to `http://localhost:8008`. Select `chat_time_weather_agent` from the top-left dropdown to interact with the agent.

Method 3: Event Lifecycle Observation Script (`run_events.py`)

To inspect the internal `Event` object flow generated by ADK's `Runner` engine (User message ➡️ Model function call request ➡️ Tool execution & result return ➡️ Model final response), run the script located in the parent `Direct_ADK` folder:

```
# Run from part02_architecture directory
uv run Direct_ADK/run_events.py
# or python Direct_ADK/run_events.py
```

Method 4: Programmatic Execution via Python Script (Runner Integration)

You can also invoke the agent programmatically by writing a simple Python script using `Runner`:

```
import asyncio
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai import types
from chat_time_weather_agent.agent import root_agent

async def main():
    # Initialize session service and runner
    session_service = InMemorySessionService()
    runner = Runner(agent=root_agent, app_name="demo_app", session_service=session_service)

    # Create a new session
    session = await session_service.create_session(app_name="demo_app", user_id="user_1")

    # Send user query
    query = "What time is it in Tokyo right now, and how is the weather?"
    msg = types.Content(role="user", parts=[types.Part(text=query)])

    print(f"[User Query]: {query}")
    async for event in runner.run_async(user_id="user_1", session_id=session.id, new_message=msg):
        if event.is_final_response():
            print(f"[Agent Response]:\n{event.content.parts[0].text}")

if __name__ == "__main__":
    asyncio.run(main())
```

---

5. Conversation Scenarios and Examples

Scenario 1: Combined Time & Weather Query (Tool Chaining)

- **User**: "What time is it in New York and how is the weather?"
- **Agent Internal Workflow**:
  1. Calls `geocode(location="New York")` ➡️ Resolves latitude `40.7128`, longitude `-74.0060`, timezone `America/New_York`
  2. Calls `get_current_time(location="New York, United States", ...)` ➡️ Computes local time
  3. Calls `get_weather(location="New York, United States", ...)` ➡️ Fetches temperature and weather conditions
- **Agent Final Output**:

> The current local time in New York, United States is 2026-11-12 20:00 EST (UTC-0500). The weather is clear, with a temperature of 12.5°C (feels like 11.8°C), humidity of 45%, and wind speed of 8.2 km/h.

Scenario 2: General Knowledge Query (Tool Bypass)

- **User**: "Summarize the mechanism of Python decorators in 2 sentences."
- **Agent Internal Workflow**:
  - Recognizes that this is not a time/weather query; bypasses all tool calls (`function_calls` empty).
- **Agent Final Output**:

> A decorator is a Python syntax feature that allows you to dynamically extend or modify the behavior of a function without changing its source code. It operates as a higher-order function that takes another function as an argument and returns a new wrapper function, typically invoked using the `@` symbol.

---

6. Troubleshooting and FAQ

Symptom	Root Cause	Resolution
`ImportError: cannot import name 'root_agent'`	Folder name contains hyphens (`-`) or `__init__.py` is missing	Verify the folder is named `chat_time_weather_agent` and that `__init__.py` exists.
`API_KEY_INVALID` or `Gemini key missing`	No valid API key configured in `.env`	Ensure `.env` contains a valid `GEMINI_API_KEY=...` .
Geocoding failure or network timeout	Open-Meteo API blocked or running offline	Set `AGENT_OFFLINE=1` in `.env` to utilize the built-in offline database.
`adk: command not found`	Google ADK CLI is not installed in the active environment	Run `pip install google-adk` and ensure your virtual environment is active.